Lab 01 — Questions

Answer without going back to the theory. One to five sentences are enough; your reasoning matters more than the exact terminology.

Question 1 [Understanding]

A colleague claims: "A container is a small virtual machine with a minimal Linux inside." What is wrong with that statement? Explain why this confusion has concrete consequences for start-up time and disk usage.

Question 2 [Analysis]

The `postgres:16-alpine` image is about 250 MB and contains a full Linux directory tree (`/bin`, `/etc`, `/usr`...). Yet we say "there is no OS in a container". Both statements are true. Explain what that image really contains, and which parts of an operating system are **missing** from it.

Question 3 [Analysis]

You start two containers from the same `nginx:alpine` image. On the host, `ps aux | grep nginx` shows two sets of processes. Yet from inside the first container, `ps` shows only its own processes. Which kernel mechanism is at work here, and why is it **not** security in the strict sense?

Question 4 [Diagnosis]

A colleague who uses Docker shows you this:

```
Client: Docker Engine - Community
 Version:  29.7.2
Cannot connect to the Docker daemon at unix:///var/run/docker.sock.
Is the docker daemon running?
```

Explain what happened on their machine (two likely causes) and why tweaking the command is pointless. Then explain why this message can **never** appear on your rootless Podman under WSL.

Question 5 [Understanding]

"An image does not run." Justify that statement, then explain what the engine concretely adds to the image the moment you run `podman run`.

Question 6 [Analysis]

You start a PostgreSQL container, create a database and some tables inside it, then remove the container with `podman rm`. You start a new container from the **same image**. Is your data still there? Justify your answer using the image/writable-layer structure — and say whether your work modified the image.

Question 7 [Analysis]

You start ten containers from the `eclipse-temurin:21-jre-alpine` image (about 210 MB). Roughly how much extra disk space does that use? Explain the mechanism that makes this answer possible.

Question 8 [Diagnosis]

Inside a rootless Podman container, `id` prints `uid=0(root)`. On the WSL host, `podman top <container> user,huser` prints `root` in the USER column and `1000` in the HUSER column. What does this double identity mean, which namespace produces it, and what can that "root" actually do if it tries to write to the host's `/etc/shadow` through a mount?

Question 9 [Understanding]

Your company forbids adding users to the `docker` group on production servers and requires everyone to go through `sudo`, with an audit trail. What is the security reasoning behind that rule, and why does rootless Podman make the whole question moot?

Question 10 [Analysis]

Convert the following commands to their long form (`podman <object> <action>`), and say for each one which **type of object** it acts on:

```
podman ps -a
podman images
podman rmi nginx:alpine
podman rm web
```

Why don't `podman ps` and `podman images` follow the same naming logic?

Question 11 [Diagnosis]

From your Ubuntu terminal under WSL, `podman run --rm alpine uname -r` prints `6.6.87.2-microsoft-standard-WSL2`. A colleague on a native Ubuntu server gets `6.8.0-45-generic` from the exact same command. Explain where each value comes from, and what this implies for the claim that "containers are lightweight" on a Windows workstation.

Question 12 [Analysis]

A badly written containerized application gets stuck in an infinite loop and eats all available RAM. Does the `pid` namespace stop it from harming the other containers? Which mechanism should you use instead, and what happens if nobody has configured it?

Question 13 [Understanding]

At work, the Spring Boot back-end image built by CI is promoted from integration to staging and then to production **without ever being rebuilt**. The staging servers run Docker; production runs Podman. Which properties make this practice possible, and what risk do you take if you rebuild the image at every stage from the same source code?

Question 14 [Analysis]

A developer adds `alias docker=podman` to their `.bashrc` and claims that "anything written for Docker will just work". Give two examples where that holds without reservation, and two situations where Podman's different architecture (no daemon, rootless) visibly changes the behavior.